

Module Summary — A Reproducible Data Workflow on the UCI Wine Quality Dataset

Author: Cameron Doelling

Overview

I built an end-to-end, reproducible data workflow in a single Jupyter notebook, `data_workflow.ipynb`, using Python, NumPy, pandas, Matplotlib and Seaborn. The notebook ingests the UCI Wine Quality dataset (<https://archive.ics.uci.edu/dataset/186/wine+quality>), cleans it with documented and reusable functions, explores it with summary statistics and correlation checks, and communicates the findings through four labelled figures and a written interpretation. No machine learning models are trained; the deliverable is the dependable data foundation that later modelling work is built on.

Dataset Description

The Wine Quality dataset records laboratory measurements and sensory evaluations for 6,497 Portuguese *Vinho Verde* wines, of which 1,599 are red and 4,898 are white. It was assembled by Cortez, Cerdeira, Almeida, Matos and Reis (2009) from wines certified between May 2004 and February 2007, and each row is one wine sample described by 13 columns. Eleven of those columns are objective physicochemical measurements taken during certification testing: fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free and total sulphur dioxide, density, pH, sulphates and alcohol. The twelfth column, `quality`, is the target variable — the median score from at least three blind sensory evaluations by wine experts, on a 0 (very bad) to 10 (excellent) scale. The thirteenth column, `color`, distinguishes red from white wine. My analysis focused on `quality` as the outcome and on `alcohol`, `volatile_acidity` and `density` as the measurements most strongly associated with it, with `color` used throughout as a grouping variable.

Workflow Description

The workflow follows five stages, each isolated in its own notebook section so that a reader can follow — and rerun — one stage at a time.

Ingestion. The raw CSV is committed to the repository beside the notebook and read with `pandas.read_csv()`. Storing the raw data alongside the analysis code, and never editing it in place, is a core recommendation for reproducible project organisation (Wilson et al., 2017); it also means the notebook runs without a network connection, so a reviewer gets the same bytes I did.

Cleaning. Four functions, each with a docstring, are applied in a fixed order through a `pandas.pipe()` chain: `standardize_columns()` normalises column and category labels, `remove_duplicate_records()` drops exact duplicate rows, `handle_missing_values()` reports and imputes gaps, and `flag_extreme_values()` marks statistical outliers without deleting them. Every function returns a new dataframe rather than mutating its input, so any step can be rerun in

isolation.

Exploratory analysis. Three EDA functions produce the evidence base: `summarize_numeric()` extends `describe()` with the median, interquartile range and skew; `profile_by_group()` compares group means across color and across each quality score; and `correlations_with()` ranks every measurement by its linear correlation with quality.

Visualization. Four figures, each titled and axis-labelled, are rendered and saved to `figures/` so that the images referenced in this report are exactly the ones the notebook produced.

Summary. A closing Markdown section states what the data showed, what surprised me, and which assumptions and limitations qualify the conclusions.

Key Decisions and Assumptions

Writing every step as a documented function. The single most consequential design decision was to express the workflow as small, docstringed functions applied in a fixed order, rather than as ad-hoc cells that mutate one dataframe. Danchev (2022) frames reproducible data science as an explicit workflow — data preprocessing, description, prediction, inference — carried out with open-source tools in literate notebooks, where the code, the reasoning and the output travel together; that framing is what shaped my notebook's structure into named workflow stages and reusable functions instead of a linear scratchpad. Combined with seeding NumPy's random generator, pinning library versions in `requirements.txt` and keeping the raw data immutable (Wilson et al., 2017), it means another person can reproduce my numbers exactly rather than approximately.

Dropping exact duplicates. The largest data-quality problem was duplication: 1,177 of 6,497 rows — 18.1% of the file — were exact copies of another row. I removed them, keeping the first occurrence. Duplicate rows carry no additional information about wine chemistry, but they do weight every mean, every correlation and every plot toward whichever wine profiles happen to repeat, so leaving them in would have quietly distorted the results reported below. This rests on an assumption worth naming: that two rows identical across all twelve measurements are repeated records of one sample rather than two distinct bottles that coincidentally match on every measurement. Given the precision of the measurements, that is the more plausible reading, but it is an assumption, not a fact about the data.

Reporting rather than imputing missing values. The dataset turned out to have no missing values, so `handle_missing_values()` reported that and imputed nothing. The function nonetheless encodes the policy I would have applied: drop any column missing more than 30% of its values, because imputing a majority of a column fabricates data rather than recovering it, and fill remaining numeric gaps with the median, which is robust to the skewed distributions typical of chemical measurements.

Flagging outliers instead of deleting them. Sixty-two rows contain a measurement more than five standard deviations from its column mean. I flagged them in a boolean column and kept them. Extreme values in wine chemistry are frequently genuine — a dessert wine really does have very

high residual sugar — so deleting them would remove real signal and bias the sample toward typical wines. Flagging keeps the decision visible and reversible for anyone reading the notebook.

What each figure was designed to show. Figure 1 shows the distribution of quality scores by colour, because the balance of the target variable determines how much weight any later claim can carry. Figure 2 is a correlation heatmap, chosen to answer two questions at once: which measurements track quality, and which measurements duplicate each other. Figure 3 is a boxplot of alcohol by quality score, which tests whether the strongest correlation is a steady trend or an artefact of sparsely populated extreme scores — a distinction a single correlation coefficient cannot make. Figure 4 plots the two leading drivers together, split by colour, to show how they combine within each wine style.

Results and Interpretation

Quality scores are heavily concentrated in the middle of the scale. Scores of 5 and 6 account for 77% of all wines, while the extremes — scores of 3 and 9 — together account for less than 1% (Figure 1). This imbalance is the single most important caveat on everything that follows: any statement about excellent or poor wines rests on a few dozen observations at most.

Alcohol content is the strongest single correlate of quality ($r \approx 0.47$), and Figure 3 shows the relationship is not an artefact of the sparse tails: median alcohol rises steadily from score 5 through score 8. The strongest negative correlates are density ($r \approx -0.33$) and volatile acidity ($r \approx -0.27$). Density's role is largely mechanical rather than sensory — it correlates -0.67 with alcohol and 0.52 with residual sugar (Figure 2), so it is substantially a restatement of a wine's sugar and alcohol content. Volatile acidity is the acetic acid that gives wine a vinegary character, and its negative association with the score is the most directly interpretable result in the analysis.

Figure 2 also exposes redundancy among the measurements that matters for any modelling built on this data: free and total sulphur dioxide correlate at 0.72 , and density is closely tied to both alcohol and residual sugar. A later model would gain little from carrying all of these.

Red and white wines form distinct chemical populations (Figure 4). White wines average more than twice the residual sugar and nearly three times the total sulphur dioxide of reds, yet their mean quality scores differ only slightly — 5.85 for whites against 5.62 for reds. Two results surprised me. The first was the sheer scale of duplication, an 18% inflation that no summary statistic would have exposed without an explicit check. The second was that the variables usually described as defining a wine's character, such as citric acid ($r \approx 0.10$) and residual sugar ($r \approx -0.06$), are close to uncorrelated with the quality score, while the plainest measurement of all, alcohol, is the best single predictor.

Finally, every relationship reported here is linear and pairwise, so curved and interacting effects are invisible to it, and none of it is causal. Raising a wine's alcohol content would not, by itself, make it taste better.

Responsible Practice (Bias and Data Quality)

The clearest place where data handling could mislead in this project is the outlier decision. Had I deleted rather than flagged the 62 extreme rows, I would have systematically removed unusual wines — the very sweet, the heavily sulphured — and produced a tidier dataset that no longer represents the population it claims to describe. Keeping them, and marking them, preserves that representativeness while still letting a later analysis exclude them deliberately.

Deduplication cuts the other way. It is the right call for summary statistics, but it is also an irreversible judgement about what a row means, so I reported the exact count removed in the notebook output rather than performing it silently. Wilson et al. (2017) argue that every manual or scripted change to raw data should be recorded as a documented, re-executable step, precisely so that decisions like this remain auditable; that is why the cleaning functions print what they changed instead of only returning a result.

Two forms of bias are inherent in the data and cannot be cleaned away. The target is *perceived* quality, the median of at least three expert tasters' scores (Cortez et al., 2009), so it encodes the preferences of a particular panel and a particular wine culture rather than an objective standard. And the sample is drawn entirely from one Portuguese region and one wine style, with whites outnumbering reds three to one — so conclusions do not transfer to wines in general, and any aggregate statistic computed across both colours is dominated by whites. Reporting results by colour, as Figures 1 and 4 do, is a partial mitigation. The remaining risk is the class imbalance in the target: were this taken forward into modelling, I would stratify any train/test split by quality score and report per-class performance rather than overall accuracy, since a model that never predicts anything but 5 or 6 would already look accurate.

Reproducibility

Anyone can rerun this work from a clone of the repository. The raw dataset is committed beside the notebook, so no download step and no network connection are required, and the exact library versions are pinned in `requirements.txt`, installable with `pip install -r requirements.txt` into a fresh virtual environment. NumPy's random seed is fixed and all plotting defaults are set in the first code cell, so a top-to-bottom run reproduces the same numbers and the same figures. The notebook is committed with its outputs intact, which lets a reviewer compare their run against mine cell by cell. Danchev (2022) makes the same argument for notebook-based teaching resources: an executable notebook plus a pinned dependency list lowers the barrier to rerunning an analysis to almost nothing.

Version control carries the history. The project is developed in the GitHub repository `ai-programming-foundations-project` with a series of commits corresponding to the stages of the workflow — project scaffolding, data ingestion, cleaning functions, EDA, visualizations, and documentation — and a development branch used for the analysis work before it was merged into `main`. Working on a branch rather than committing directly to `main` keeps the trunk in a state that always runs, and the commit history shows how the analysis was arrived at, not merely where it ended up (Wilson et al., 2017).

References

- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547–553. <https://doi.org/10.1016/j.dss.2009.05.016>
- Danchev, V. (2022). Reproducible Data Science with Python: An Open Learning Resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- The pandas development team. (2024). *pandas documentation: Working with missing data*. https://pandas.pydata.org/docs/user_guide/missing_data.html
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>

Appendix: Figures Referenced in This Report

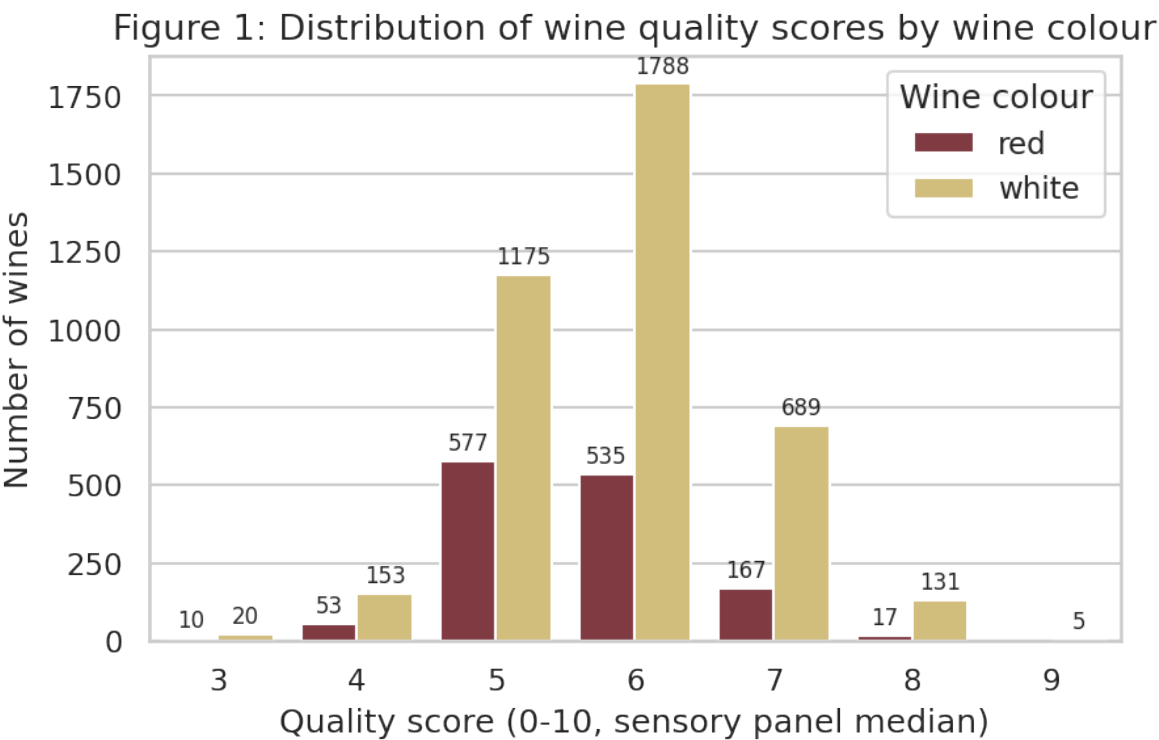


Figure 1: Distribution of wine quality scores by wine colour.

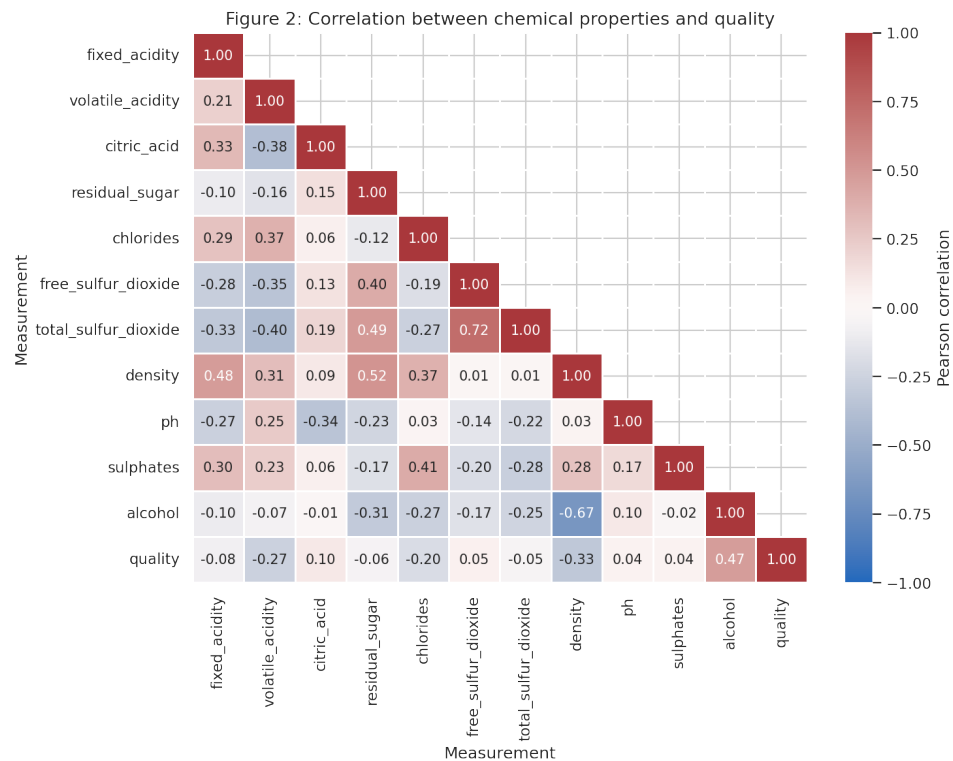


Figure 2: Correlation between chemical properties and quality.

Figure 3: Alcohol content rises with wine quality score

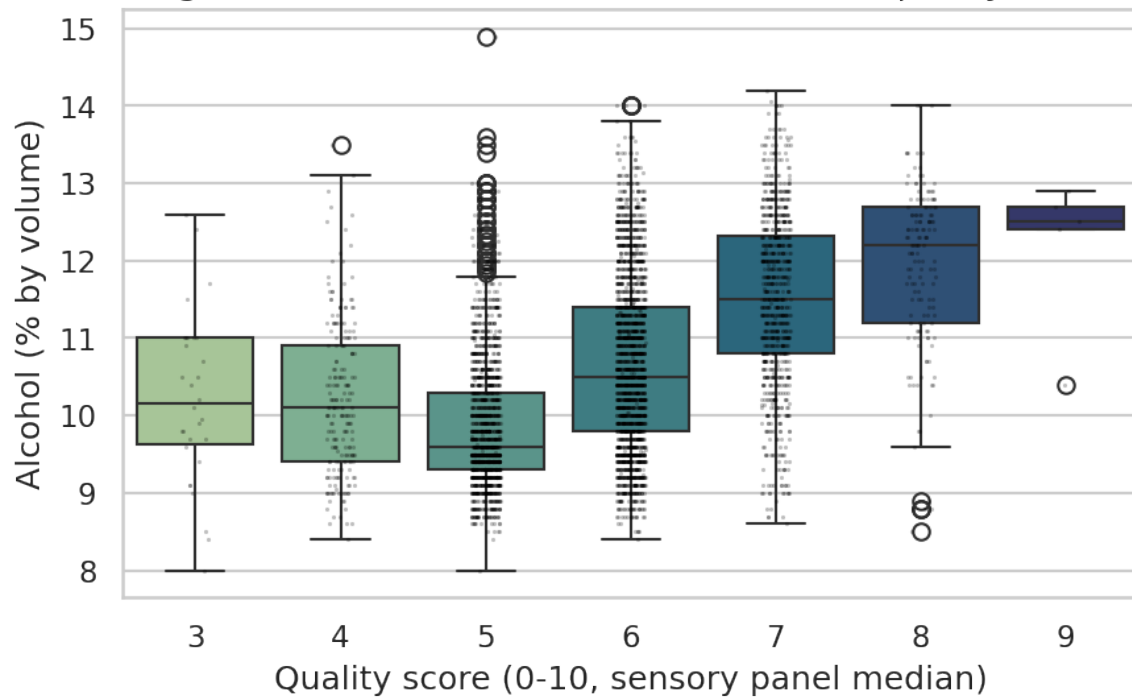


Figure 3: Alcohol content rises with wine quality score.

Figure 4: Higher alcohol and lower volatile acidity accompany better scores
Red wines (n = 1,359) White wines (n = 3,961)

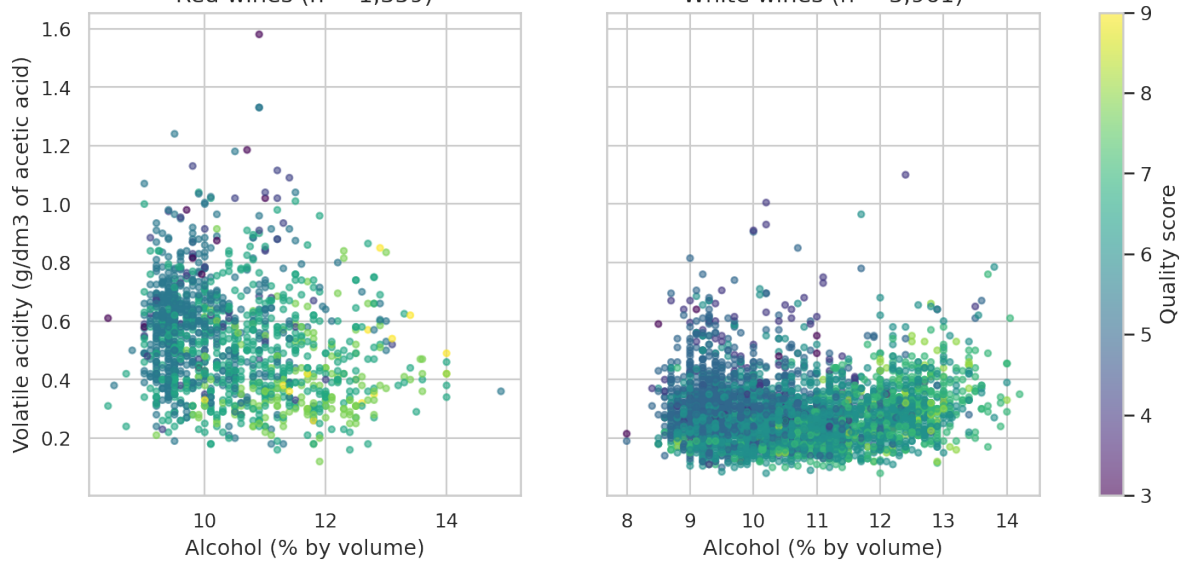


Figure 4: Higher alcohol and lower volatile acidity accompany better scores.